



Cloud-native · Event-driven · Kubernetes-orchestrated

Node.js microservices · SQLite per-service · Docker · Kubernetes (Minikube)

*Source: commit **eddd9d5** · github.com/rajasekar21/ride-hailing-platform*

TEAM MEMBERS

- Akanksha Rathore
- Manesh Nivas Borkar
- N K Anil Kumar
- Saif Uddin Ahamed
- Rajasekar S

Team-26 · 2025

Table of Contents

1. System Overview	3
2. Architecture & Data Design	3
3. Tech Stack	3
4. Run Options	3
5. API Versioning & Endpoints	4
6. Business Rule Verification	4
7. Monitoring, Metrics & Logs	5
8. Documentation Artifacts	5
9. Evidence Mapping	5
10. Standalone Service Repos	6
11. Task Distribution	6
12. Contributors	6



1. System Overview

Cloud-native ride-hailing platform built with Node.js microservices, SQLite database-per-service, Docker, and Kubernetes (Minikube). Structured for assignment evaluation with code, deployment manifests, API artifacts, and screenshot/video evidence.

Core Flow

- Rider → Trip → Driver → Payment → Notification → Rating

Implemented Services

- services/user — Rider management
- services/driver — Driver onboarding + active state
- services/ride — Trip lifecycle + fare + orchestration
- services/payment — Charge/refund, idempotency, rate limiting
- services/notification — Event consumption + logs
- services/rating — Post-trip rating
- services/auth — JWT login

2. Architecture & Data Design

- Microservices architecture with loose coupling
- Database-per-service pattern (SQLite per service)
- Synchronous HTTP inter-service communication (axios)
- Asynchronous events via RabbitMQ (amqpLib)

SQLite Note

SQLite is used intentionally for assignment simplicity. SQLite-backed deployments remain single-replica to avoid DB-file multi-writer conflicts.

3. Tech Stack

Layer	Technology
Backend	Node.js, Express
Frontend	React (Vite)
Data	SQLite
Messaging	RabbitMQ
Containerization	Docker
Orchestration	Kubernetes (Minikube)
Monitoring	Prometheus, Grafana

4. Run Options

Docker Compose

```
docker compose up -d --build
```

Frontend: <http://localhost:5173>

Kubernetes (Minikube)

```
minikube start --driver=docker --memory=4096 --cpus=2
kubectl apply -f k8s/
kubectl wait --for=condition=ready pod --all --timeout=240s
kubectl get pods
kubectl get svc
```

5. API Versioning & Core Endpoints

All core routes are versioned under /v1.

Riders

- GET/POST /v1/riders
- GET/PUT/DELETE /v1/riders/{id}

Drivers

- GET/POST /v1/drivers
- GET /v1/drivers/{id}
- PATCH /v1/drivers/{id}/status

Trips

- POST/GET /v1/trips | GET /v1/trips/{id}
- POST /v1/trips/{id}/accept|complete|cancel

Payments

- POST /v1/payments/charge | GET /v1/payments
- GET /v1/payments/{id} | POST /v1/payments/{id}/refund

Ratings

- POST /v1/trips/{id}/rating
- GET /v1/ratings | GET /v1/ratings/trip/{tripId}

6. Required Business Rule Verification

1) Fare Formula + Surge

File: services/ride/app.js

```
rg -n "function calculateFare|surgeOptions|randomSurge|Math.round|ratePerKm|baseFare" services/ride/app.js
```

- surgeOptions = [1.0, 1.2, 1.5]
- Fare formula: baseFare + distance * ratePerKm * surge
- Rounded to 2 decimals: Math.round(... * 100) / 100

2) Active Driver Enforcement Before Accept

File: services/ride/app.js

```
rg -n "v1/trips/:id/accept|is_active|Driver is not active|status\{422\}" services/ride/app.js
```

- Driver active state is checked before accept
- Inactive driver returns 422 with "Driver is not active"

3) Payment Idempotency + Rate Limiting

File: services/payment/index.js | Evidence: docs/evidence/PAYMENT_SOURCE_VERIFICATION.md

```
rg -n "Idempotency-Key|idempotency_keys|rateLimit|max:\s*10|Too many requests|/payments/charge" services/payment/index.js
```

- Missing Idempotency-Key header returns 400
- Duplicate key returns stored response (no reprocessing)
- idempotency_keys SQLite table present
- Rate limiter on charge: 10/min per IP, 429 on exceed

7. Monitoring, Metrics & Logging

- Prometheus config: monitoring/prometheus.yml
- Compose wiring: docker-compose.yml
- Metrics endpoint: GET /metrics (Prometheus format)

Required Metrics

- trips_requested_total
- trips_completed_total
- payments_failed_total
- avg_driver_rating

Structured logs include correlation IDs (x-correlation-id propagation).

Verification Commands

```
curl http://localhost:3000/metrics
curl http://localhost:3003/metrics
curl http://localhost:3005/metrics
kubectl logs deployment/notification --since=10m
```

8. Documentation Artifacts

- docs/README.md
- docs/api/API_DOCUMENTATION.md
- docs/api/postman_collection.json
- docs/api/openapi.json
- docs/architecture/ER_DIAGRAM.md | docs/architecture/ER_DATA_MODEL.md
- docs/evidence/DEPLOYMENT_EVIDENCE.md
- docs/evidence/PAYMENT_SOURCE_VERIFICATION.md
- docs/screenshots/README.md

OpenAPI Generation

```
cd docs/api && npm run generate:openapi
# CI check: .github/workflows/openapi-verify.yml
```

9. Evidence Mapping

All evidence assets are under docs/screenshots/.

Asset	Evidence
evidence-docker-ps.png	docker ps
evidence-kubectl-get-pods.png	kubectl get pods
evidence-kubectl-get-svc.png	kubectl get svc
evidence-k8s-probes-check.png	Probe presence (readinessProbe livenessProbe)
evidence-metrics-*.png	Service metrics outputs
evidence-logs-notification-json.png	Structured log proof
videos/workflow-demo-playwright.webm	Demo workflow video

10. Standalone Service Repository Links

Platform orchestration: github.com/rajasekar21/ride-hailing-platform
User service: github.com/rajasekar21/ride-hailing-user-service
Driver service: github.com/rajasekar21/ride-hailing-driver-service
Trip service: github.com/rajasekar21/ride-hailing-trip-service
Payment service: github.com/rajasekar21/ride-hailing-payment-service
Rating service: github.com/rajasekar21/ride-hailing-rating-service
Notification service: github.com/rajasekar21/ride-hailing-notification-service
Auth service: github.com/rajasekar21/ride-hailing-auth-service

11. Task Distribution

Primary ownership and contribution area for each team member.

#	Contributor	Tasks & Responsibilities
1	Akanksha Rathore	User Service (REST API, SQLite schema, seed data), React frontend dashboard, Leaflet map integration
2	Manesh Nivas Borkar	Driver Service (onboarding, active state, RabbitMQ consumer), Docker image builds, Kubernetes manifests
3	N K Anil Kumar	Ride Service (trip lifecycle, fare/surge formula, orchestration), dataset CSV generation, DB seeding
4	Saif Uddin Ahamed	Payment Service (idempotency, rate limiting, charge/refund), Notification Service, RabbitMQ & Redis setup
5	Rajasekar S	Auth Service (JWT, RBAC), Rating Service, HPA config, Prometheus/Grafana, CI/CD scripts, architecture & README

12. Contributors

1	2	3	4	5
Akanksha Rathore	Manesh Nivas Borkar	N K Anil Kumar	Saif Uddin Ahamed	Rajasekar S

github.com/rajasekar21/ride-hailing-platform · Commit eddd9d5